Department of Information Technology

LAB MANUAL

Quantum Algorithms and Computational Methods

(23UQCMDL3503)

(2023 PATTERN)

INSTITUTE VISION AND MISSION

VISION

To achieve excellent standards of quality education by keeping pace with rapidly changing technologies and create technical manpower of global standards with capabilities of accepting new challenges.

MISSION

Our efforts are dedicated to impart quality and value-based education to raise satisfaction level of all stake- holders. Our strength is directed to create competent professionals. Our endeavor is to provide all possible support to promote research and development activities.

DEPARTMENT VISION AND MISSION

VISION

To achieve excellence in the field of Information Technology by developing Competent Engineers to cater the need of IT industry and Society.

MISSION

The department strives to:

M1: Achieve excellence in teaching learning process by imparting quality and value-based education

M2: Groom competent IT professionals by keeping pace with changing technology to address needs of industry and society

M3: Promote research and innovation

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Educational Objectives (PEOs)

PEO1: To prepare the graduates to apply engineering knowledge to design and develop feasible solutions to complex problems

PEO2: To prepare the graduates to engage in research and innovation to solve societal and Industrial problems

PEO3: To prepare the graduates to exhibit ethics and professionalism to work effectively in multidisciplinary team

Program Specific Outcomes (PSOs)

PSO1: Graduate will possess knowledge of IT Fundaments, Web Technology, Database Management Systems, and Software Engineering to develop innovative applications

PSO2: Graduate will have an ability to develop multidisciplinary applications to cater societal needs by using Modern IT tools & Technologies

G H Raisoni College of Engineering & Management, Pune
Second Year of Information Technology (2023 Pattern)
23UQCMDL3503: Quantum Algorithms and Computational Methods

Teaching Scheme:	**Credit**	**Examination Scheme:**
PR: 02 Hours/Week	**01**	**Ext: 25 Marks**
		Total: 25 Marks

Sr. No.	**List of Laboratory Assignments.**
1	Create a single-qubit circuit applying the Hadamard gate
2	Measure the qubit and plot the histogram of results
3	Build a circuit to generate a Bell state
4	Submit a Bell state circuit to a quantum simulator (IBM)
5	Implement a multi-qubit circuit with CNOT and SWAP gates
6	Implement and run the Deutsch-Jozsa algorithm on a balanced and constant function
7	Implement and run Grover's algorithm to search for a marked state
8	Implement and run Quantum Fourier Transforms and Shor's algorithm

Course Objectives:

1. Introduce core quantum computing concepts including qubits, superposition, and mathematical foundations.
2. Explain quantum mechanics principles and their application in gates, circuits, and measurements.
3. Develop understanding of key quantum algorithms like Deutsch-Jozsa, Grover's, and Shor's.
4. Explore practical implementations and applications of quantum computing in real-world domains.

Course Outcome:

After completion of this course students will be able to:

CO1: Understand the foundational principles of quantum mechanics and foundations of quantum computing.

CO2: Implement fundamental quantum algorithms.

CO3: Interpret various quantum-classical practices.

CO4: Comprehend the applications of quantum computing.

CO and PO Mapping with practical

Sr. No.	Experiment List	Mapping With CO	Mapping With PO & PSO
1	Create a single-qubit circuit applying the Hadamard gate.	CO1	PO1, PO2, PO4, PO5, PO8, PO10, PO12, PSO2
2	Measure the qubit and plot the histogram of results.	CO1	PO1, PO2, PO4, PO5, PO8, PO10, PO12, PSO2
3	Build a circuit to generate a Bell state.	CO2	PO1, PO2, PO4, PO5, PO8, PO10, PO12, PSO2
4	Submit a Bell state circuit to a quantum simulator (IBM).	CO2	PO1, PO2, PO4, PO5, PO8, PO10, PO12, PSO2
5	Implement a multi-qubit circuit with CNOT and SWAP gates.	CO3	PO1, PO2, PO4, PO5, PO8, PO10, PO12, PSO2
6	Implement and run the Deutsch-Jozsa algorithm on a balanced and constant function.	CO3	PO1, PO2, PO4, PO5, PO8, PO10, PO12, PSO2
7	Implement and run Grover's algorithm to search for a marked state.	CO4	PO1, PO2, PO4, PO5, PO8, PO10, PO12, PSO2
8	Implement and run Quantum Fourier Transforms and Shor's algorithm.	CO4	PO1, PO2, PO4, PO5, PO8, PO10, PO12, PSO2

CO Wise Weightage and Practical Marks

Marks of Each Experiment: 10

Total Marks of 8 Experiments: 80

Total Practical Marks: 25

Expt. No.	**Mapped CO**	**Marks of**			
		CO1	CO2	CO3	CO4
01	CO1	10			
02	CO1	10			
03	CO2		10		
04	CO2		10		
05	CO3			10	
06	CO3			10	
07	CO4				10
08	CO4				10
Weightage out of 80		20	20	20	20
Weightage out of 25		***6***	***6***	***6***	***7***

Experiment No. 01

Create a single-qubit circuit applying the Hadamard gate

Aim: Design and Simulation of Quantum Circuits using Single-Qubit Gates in Qiskit

Objective:

- To understand and implement quantum logic gates using Qiskit.
- To design circuits using Single-qubit gates: X, Y, Z, H.
- To simulate these circuits.

Software Required:

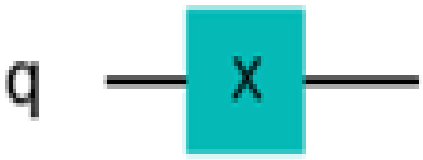
1. Python 3.x installed
2. Qiskit installed: pip install qiskit
3. Jupyter Notebook (recommended)

Theory:

Single-Qubit Gates: Single qubit inputs are $|0\rangle$ or $|1\rangle$ Single Qubit Gates are X- gate, Y-gate, Z- gate, H –gate, S-gate, T-gate.

1. **Pauli-X Gate (X Gate):**
   X-gate is the single qubit input gate and is also called as Pauli X – gate or quantum NOT gate. The Matrix form of X is given by

$$\sigma_X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = |0\rangle\langle 1| + |1\rangle\langle 0|$$



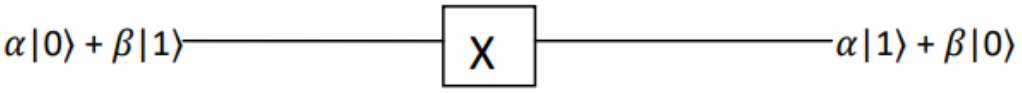
When X operates on $|0\rangle$ and $|1\rangle$ the output will be inverted (ie,$|0\rangle$ becomes $|1\rangle$ and $|1\rangle$ becomes$|0\rangle$)that is :

$$X|0\rangle=|1\rangle$$

$$X|1\rangle=|0\rangle$$

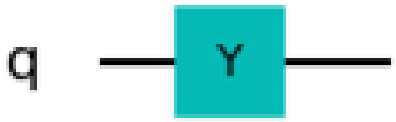
Let the input be a superposed qubit: $|\psi\rangle=\alpha|0\rangle+\beta|1\rangle$ Then, the action of the X-gate is:

$X|\psi\rangle=\alpha|1\rangle+\beta|0\rangle$ It **swaps the amplitudes** of $|0\rangle$ and $|1\rangle$.



2. **Y-Gate (Pauli-Y Gate):**
   The Y-gate is a single-qubit quantum gate that combines a bit-flip and phase-flip operation. It is one of the Pauli matrices and introduces a phase of i or −i during transformation

$$\sigma_Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} = i(|1\rangle\langle 0| - |0\rangle\langle 1|)$$



$Y|0\rangle=i|1\rangle$

$Y|1\rangle=-i|0\rangle$

This shows that **Y** flips the state and adds a phase factor: $|0\rangle$ becomes $\mathbf{i\cdot|1\rangle}$ and $|1\rangle$ becomes $\mathbf{-i\cdot|0\rangle}$

Let: $|\psi\rangle=\alpha|0\rangle+\beta|1\rangle$

Then: $Y|\psi\rangle=\alpha\cdot i|1\rangle+\beta\cdot(-i)|0\rangle=-i\beta|0\rangle+i\alpha|1\rangle$



3. **Z-Gate (Pauli-Z Gate / Phase-Flip Gate):**
   The Z-gate is a single-qubit quantum gate that flips the phase of the $|1\rangle$ state. It is also called the phase-flip gate because it multiplies $|1\rangle$ by -1 while leaving $|0\rangle$ unchanged.

$$\sigma_Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = |0\rangle\langle 0| - |1\rangle\langle 1|$$



$Z|0\rangle=|0\rangle$

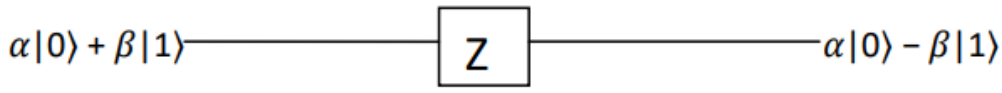
$$Z|1\rangle = -|1\rangle$$

This shows that the Z gate **does not change the probability amplitude**, but flips the **sign (phase)** of the $|1\rangle$ component.

Let: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

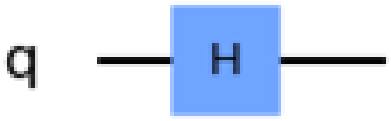
Then: $Z|\psi\rangle = \alpha|0\rangle - \beta|1\rangle$

So, the Z gate **inverts the phase** of the $|1\rangle$ component in the superposition.



4. **Hadamard Gate (H-Gate):**
   The **Hadamard gate** is a single-qubit quantum gate that transforms a basis state into an **equal superposition** of $|0\rangle$ and $|1\rangle$. It plays a crucial role in quantum parallelism and interference.

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| - |1\rangle\langle 1|)$$



$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Let:

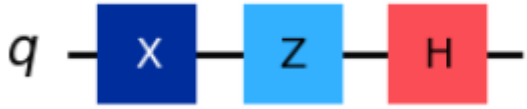
$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

Then:

$$H|\psi\rangle = \alpha \cdot \frac{(|0\rangle + |1\rangle)}{\sqrt{2}} + \beta \cdot (|0\rangle - |1\rangle)2 = (\alpha+\beta)|0\rangle + (\alpha-\beta)|1\rangle$$

Design Circuit using above gates:

The quantum circuit constructed in this experiment is a **single-qubit circuit** that sequentially applies three fundamental quantum gates — **X**, **Z**, and **H** — to demonstrate the effects of unitary operations on a qubit.



Algorithm:

1. Start with an initial quantum state, e.g., |0⟩.
2. Apply the X (NOT) gate: This flips the qubit from |0⟩ to |1⟩ or vice versa.
3. Apply the Z gate: This adds a phase flip to the qubit.
4. Apply the H (Hadamard) gate: This puts the qubit into superposition.
5. Measure the quantum state to observe the result.

Conclusion:

Experiment No. 02

Measure the qubit and plot the histogram of results

Aim: To construct, simulate, and visualize the quantum state $(|01\rangle+|10\rangle)/\sqrt{2}$ using Qiskit and understand the behaviour of quantum entanglement through measurement statistics and state vector visualization.

Objective:

- Understand how to construct an entangled quantum state using Hadamard and CNOT gates.
- Use Qiskit to simulate and measure the quantum state.
- Analyze measurement statistics using histograms.
- Visualize the statevector using Bloch sphere, Hinton diagram, city view, Pauli vector, and Qsphere.

Software Required:

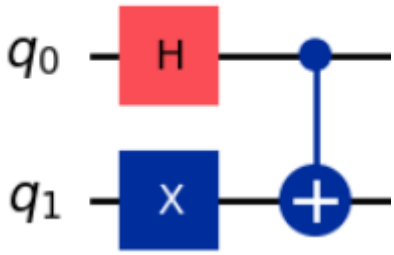
1. Python 3.x installed
2. Qiskit installed: pip install qiskit
3. Jupyter Notebook (recommended)

Theory:

Quantum entanglement is a fundamental property of quantum systems in which the state of one qubit is dependent on the state of another, no matter the distance between them. A notable example of an entangled state is:

$$|\psi\rangle = (|01\rangle+|10\rangle)/\sqrt{2}$$

This state implies that measuring the first qubit to be $|0\rangle$ forces the second to be $|1\rangle$, and vice versa. Such states are called **Bell States**, which are crucial for quantum teleportation, quantum key distribution, and superdense coding.



Circuit Explanation:

1. Start with the state $|00\rangle$.
2. Apply an **X gate** to qubit 1: changes state to $|01\rangle$.

3. Apply a **Hadamard gate (H)** to qubit 0: superposes it.
4. Apply a **CNOT gate** with control qubit 0 and target qubit 1: creates entanglement.
5. Measure both qubits to observe correlated outcomes.

Quantum State Visualization in Qiskit

When simulating quantum circuits, it is important not only to obtain measurement outcomes but also to understand the **quantum state** before measurement. Qiskit provides powerful tools to **simulate**, **measure**, and **visualize** quantum states, especially for debugging, interpretation, and learning purposes.

1. Histogram Visualization (plot_histogram)

The function plot_histogram() is used to plot the classical measurement results of quantum circuits. It offers several customization options:

- **legend**: A list of strings labeling the execution results when comparing multiple histograms (e.g., first vs. second run).
- **sort**: Controls the sorting of bars by probability; 'asc' for ascending or 'desc' for descending.
- **number_to_keep**: Displays only the top N results; all others are grouped into a single "rest" bar.
- **color**: A single color or a list of colors to style each dataset in the histogram.
- **bar_labels**: Enables or disables the labels above bars showing exact counts or percentages.
- **figsize**: Tuple to set the size of the output figure in inches.

This function returns a matplotlib.Figure, which can be used to:

- Show the figure using .show() (especially useful outside Jupyter),
- Save the figure using .savefig('filename.png').

2. Quantum State Representations

Quantum states can be represented as:

- **Statevector $|\psi\rangle$**: A pure quantum state represented as a complex vector.
- **Density matrix $\rho=|\psi\rangle\langle\psi|$** : A more general representation that also includes *mixed* states formed by a probabilistic mixture of pure states:

$$\rho = \sum_k p_k |\psi_k\rangle\langle\psi_k|$$

3. Visualization Functions for Quantum States

Qiskit provides several built-in functions to visualize quantum states:

Function	Description
plot_state_city()	Visualizes the real and imaginary parts of the density matrix as 3D bars (like a city skyline). Useful for understanding amplitude contributions and coherence.
plot_state_qsphere()	Unique spherical visualization. Amplitude is represented by arrow thickness; phase is shown via color. Useful to understand entanglement and phase relationships.
plot_state_paulivec()	Represents the quantum state in the Pauli operator basis. Helpful to see quantum operations in terms of Pauli components.
plot_state_hinton()	Similar to city view, but shows matrix values using size of squares. Larger squares indicate higher magnitude elements.
plot_bloch_multivector()	Plots each qubit's reduced state on a Bloch sphere. Useful for visualizing individual qubit states in a multi-qubit system.

Algorithm:

1. **Import Libraries**: Import Qiskit components and visualization tools.
2. **Initialize Circuit**: Create a quantum circuit with 2 qubits and 2 classical bits.
3. **Prepare Entangled State**:
   - Apply X gate to qubit 1.
   - Apply H (Hadamard) gate to qubit 0.
   - Apply CNOT with control 0 and target 1.
4. **Measurement**: Measure both qubits to classical bits.
5. **Simulation**: Use AerSimulator or statevector_simulator to simulate and collect results.
6. **Visualization**:
   - Plot histogram of measurement outcomes.
   - Plot statevector using plot_state_city, plot_state_hinton, plot_state_qsphere, plot_state_paulivec.

Conclusion:

Experiment No. 03

Build a circuit to generate a Bell state

Aim: To design, simulate, and visualize a 2-qubit Bell state using Qiskit, and analyze its entangled properties using measurement results and statevector visualizations.

Objective:

- Construct the entangled Bell state $\frac{1}{\sqrt{2}}(|00\rangle+|11\rangle)$.
- Simulate the quantum circuit using Qiskit's `AerSimulator`.
- Measure the outcome of the quantum circuit and analyze the results.
- Visualize the quantum state using multiple visualization tools in Qiskit.

Software Required:

1. Python 3.x installed
2. Qiskit installed: pip install qiskit
3. Jupyter Notebook (recommended)

Theory:

Introduction to Quantum Entanglement:

Quantum entanglement is a fundamental property of quantum mechanics where two or more particles become linked in such a way that the state of one particle instantly influences the state of the other(s), no matter the distance between them. In quantum computing, entanglement allows qubits to exhibit correlations that are impossible in classical systems, enabling exponential computational power in some algorithms.

Entangled states cannot be expressed as the product of the individual states of each qubit. Instead, their combined state must be described as a whole, indicating interdependence.

Bell State:

Bell states are the simplest and most well-known examples of entangled states. There are four canonical Bell states, each representing a unique kind of entanglement between two qubits:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$
$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$$
$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$$
$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

These states are:

- Maximally entangled (perfect correlation).
- Orthogonal and form a complete basis for 2-qubit space (Bell basis).
- Inseparable; measurement of one qubit instantaneously determines the outcome of the other.

In this experiment, we created the |Φ+⟩ state.

Bell states are the simplest examples of maximally entangled quantum states involving two qubits. In this experiment, we focus on the first Bell state:

$$|\Phi+\rangle = \frac{1}{\sqrt{2}}(|00\rangle+|11\rangle)$$

This state implies that both qubits are perfectly correlated—measuring one qubit gives an immediate knowledge of the other.

How to Create a Bell State:

1. Initialize the two-qubit state to |00⟩.
2. Apply a Hadamard gate (H) to qubit 0 to create a superposition:
   $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle+|1\rangle)$
3. Apply a CNOT gate with qubit 0 as control and qubit 1 as target to entangle the two qubits.

$$CNOT\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle\right) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

4. Measure both qubits.

Circuit Explanation:

1. Start with the state |00⟩.

2. Apply a **Hadamard gate (H)** to qubit 0: superposes it.
3. Apply a **CNOT gate** with control qubit 0 and target qubit 1: creates entanglement.
4. Measure both qubits to observe correlated outcomes.

Algorithm:

1. **Import Libraries**: Import Qiskit components and visualization tools.
2. **Initialize Circuit**: Create a quantum circuit with 2 qubits and 2 classical bits.
3. **Prepare Bell State**:
   - Apply H (Hadamard) gate to qubit 0.
   - Apply CNOT with control 0 and target 1.
4. **Measurement**: Measure both qubits to classical bits.
5. **Simulation**: Use AerSimulator or statevector_simulator to simulate and collect results.
6. **Visualization**:
   - Plot histogram of measurement outcomes.
   - Plot statevector using plot_state_city, plot_state_hinton, plot_state_qsphere, plot_state_paulivec.

Conclusion:

Experiment No. 04

Superdense Coding using Qiskit

Aim: To implement the Superdense Coding protocol using Qiskit and simulate it on a quantum circuit to verify the correct transfer of two classical bits using one qubit.

Objective:

- Learn how to encode two classical bits using quantum gates.
- Simulate the Superdense Coding protocol on Qiskit.
- Visualize outcomes using histograms.

Software Required:

1. Python 3.x installed
2. Qiskit installed: pip install qiskit
3. Jupyter Notebook (recommended)

Theory:

Superdense coding is a quantum communication protocol that allows a sender to send two classical bits of information to another user by only utilizing one qubit.
Superdense coding was first proposed by Charles Bennett and Stephen Wiesner in 1992 and experimentally actualized four years later, in 1996.
Now, it is important to note that while Superdense Coding and Quantum Teleportation are closely related, there is a key difference between them. Quantum teleportation is a process by which a user can transmit one qubit using two classical bits whereas Superdense Coding is a process by which a user can transmit two classical bits using one qubit. Basically, Superdense Coding can be thought of as the flipped version of Quantum Teleportation.
Superdense coding involves three parties, let's just call them Charlie, Alice and Bob. It requires two parties who wish to communicate a two-bit message, a pair of entangled qubits, and a quantum channel.

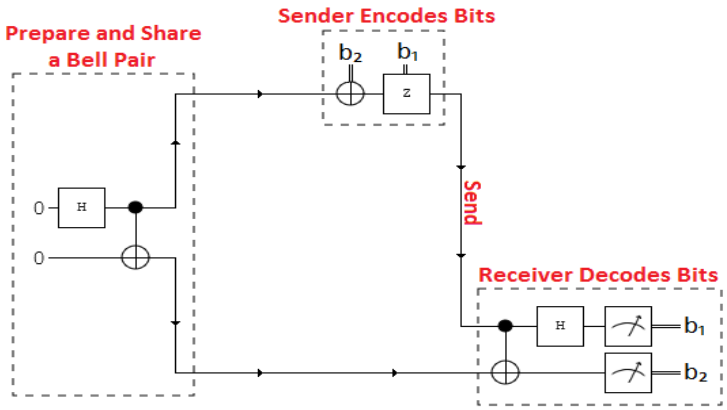


Diagram of Protocol

Charlie prepares a pair of Bell state, or maximally entangled, qubits, which just means that the two qubits exhibit perfect correlations even though they are spatially seperated (even with great distances).
Charlie then sends those qubits to Alice and Bob so that they can share them between themselves (1 qubit with Alice and the other with Bob). However, before Alice can attempt to transmit 2 classical bits of infortmation to Bob using her Bell state qubit, she has to apply a single gate operation based off of the intended message:

Intended Message	Applied Gate	Resulting State ($\cdot\sqrt{2}$)
00	I	$\|00\rangle + \|11\rangle$
10	X	$\|01\rangle + \|10\rangle$
01	Z	$\|00\rangle - \|11\rangle$
11	ZX	$-\|01\rangle + \|10\rangle$

Now, Bob is a very smart guy. He knows that since Alice's qubits are entangled the two qubits must be in one of the four Bell states. So, once Bob recieves Alice's encoded qubit, he will pass it and his untouched Bell state qubit through a reverse Bell circuit (where Alice's qubit acts as the control and Bob's is the target) in order to decode Alice's message. Thus, Bob will obtain two classical bits of information from a single qubit!

Bob Receives:	After CNOT-gate:	After H-gate:
$\|00\rangle + \|11\rangle$	$\|00\rangle + \|01\rangle$	$\|00\rangle$
$\|01\rangle + \|10\rangle$	$\|11\rangle + \|10\rangle$	$\|10\rangle$
$\|00\rangle - \|11\rangle$	$\|00\rangle - \|01\rangle$	$\|01\rangle$
$-\|01\rangle + \|10\rangle$	$-\|11\rangle + \|10\rangle$	$\|11\rangle$

Assignment:

Alice and Bob share the Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Alice wants to send **one of the classical 2-bit messages** using Superdense Coding.

For each of the following messages: `00`, `01`, `10`, and `11`, do the following:

Questions (Repeat for each message):

1. **Draw the quantum circuit** representing the Superdense Coding protocol for this message:

   a. Include entanglement setup, Alice's operation, qubit transfer, and Bob's measurement.
2. (a) **Which gate(s)** does Alice apply to her qubit to encode the message?
3. (b) **Write the new joint quantum state** (after Alice's operation).
4. (c) **What Bell state** will Bob receive and decode?

Algorithm:

1. **Import Required Libraries:**Import qiskit and relevant visualization/simulation modules.
2. **Initialize Quantum Circuit:**Create a 2-qubit, 2-classical-bit quantum circuit.
3. **Create Bell State (Entanglement Setup):**
   - Apply Hadamard gate H on qubit 0 (Alice's qubit).
   - Apply CNOT gate with control qubit 0 and target qubit 1 (Bob's qubit).
   - This creates the entangled Bell state $|\Phi+\rangle$
4. **Encode Alice's Message:**Based on the two-bit message:
   - 00: Do nothing.
   - 01: Apply Z gate on qubit 0.
   - 10: Apply X gate on qubit 0.
   - 11: Apply Z then X on qubit 0.
5. **Send Qubit to Bob:**(Simulated) Alice sends her qubit to Bob (no gate here; logical handoff).
6. **Bob Decodes the Message:**
   - Apply CNOT gate (control: qubit 0, target: qubit 1).
   - Apply Hadamard gate on qubit 0.
7. **Measure the Qubits:**Measure both qubits into classical bits.
8. **Simulate the Circuit:**
   - Use AerSimulator to run the circuit with 1024 shots.
   - Display measurement counts and histogram.
9. **Repeat for All Messages (Optional Loop):** Loop over all messages (00, 01, 10, 11) to test full protocol.

Conclusion:

Experiment No. 05

Implement a multi-qubit circuit of SWAP Using CNOT

Aim: To implement the SWAP gate using three CNOT gates in Qiskit and simulate it on a quantum circuit to verify correct swapping of qubits.

Objective:

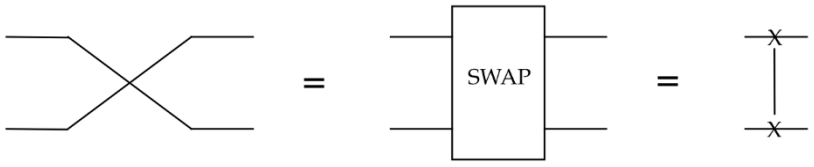
- Learn the decomposition of a SWAP gate into elementary CNOT gates.
- Implement and simulate the SWAP operation using Qiskit.
- Visualize outcomes using circuit diagrams and measurement histograms.

Software Required:

1. Python 3.x installed
2. Qiskit installed: pip install qiskit
3. Jupyter Notebook (recommended)

Theory:

The SWAP gate:



- As the name indicates, this gate swaps the *states* of two qubits.
- What's important to note:
  - The two qubits themselves don't move.
  - Instead, one's state becomes the other's state.
  - Since there are no "wires" connecting the qubits, it's not obvious this is even feasible.
- In matrix and Dirac forms:

$$\text{SWAP}=|00\rangle\langle 00|+|01\rangle\langle 10|+|10\rangle\langle 01|+|11\rangle\langle 11|$$

$$SWAP = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Assignment:

1. Draw and prove all possible the quantum circuit representing SWAP using three CNOT gates.
2. Prepare input states |01⟩, |10⟩, and |11⟩, apply the SWAP construction, and verify the outputs.

Algorithm:

1. **Import Required Libraries:** Import qiskit and relevant simulation modules.
2. **Initialize Quantum Circuit:** Create a 2-qubit quantum circuit with 2 classical bits.
3. **Prepare Input State**: Initialize qubits in specific states (|01⟩,|10⟩,|11⟩).
4. **Apply SWAP Decomposition:**
   - Apply CNOT(q0 → q1)
   - Apply CNOT(q1 → q0)
   - Apply CNOT(q0 → q1)
5. Measure both qubits into classical bits.
6. Simulate Circuit: Run the circuit using AerSimulator with 1024 shots.
7. Display results with plot_histogram.
8. Repeat for different initial states to verify correctness.

Conclusion:

Experiment No. 06

Implement and run the Deutsch–Jozsa algorithm on a balanced and constant function

Aim: To implement the Deutsch–Jozsa algorithm using Qiskit and simulate it for both a constant and a balanced oracle in order to determine whether the given oracle function is constant or balanced.

Objective:

- Understand the Deutsch–Jozsa problem and algorithmic steps.
- Construct oracle circuits for a constant function and a balanced function.
- Implement the Deutsch–Jozsa algorithm in Qiskit and run simulations.
- Analyze measurement results to distinguish constant vs balanced oracles.

Software Required:

1. Python 3.x installed
2. Qiskit installed: pip install qiskit
3. Jupyter Notebook (recommended)

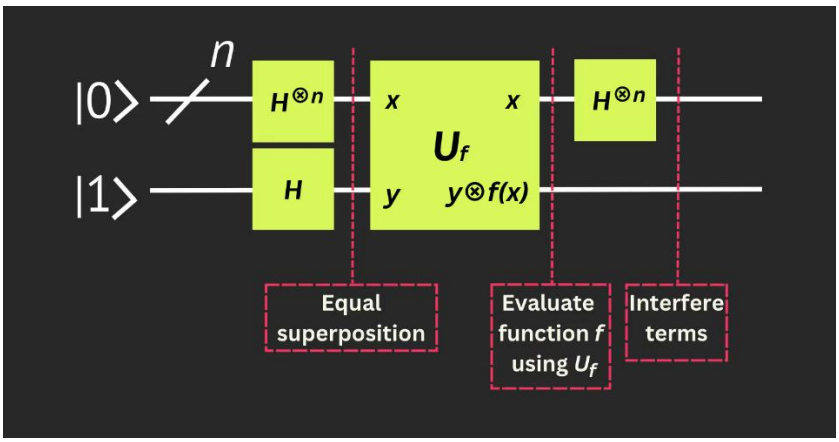
Theory:

The Deutsch–Jozsa problem: Given a black-box function (oracle)

$$f : \{0,1\}^n \rightarrow \{0,1\}$$

promised to be either **constant** (same output for all inputs) or **balanced** (outputs 0 for exactly half the inputs and 1 for the other half), determine which type the function is.

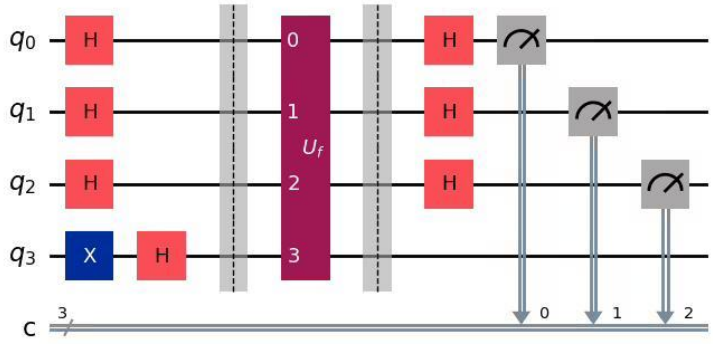
Classically, in the worst case one needs $2^{n-1}+1$ queries to be certain. The Deutsch–Jozsa quantum algorithm uses quantum parallelism and interference to determine the answer with a **single** query to the oracle (and some additional gates), giving an exponential separation in the query model.



High level steps (for n-bit input):

1. Prepare n input qubits in state $|0\rangle^{\otimes n}$ and one ancilla qubit in state |1⟩.
2. Apply Hadamard gates to all qubits to create an equal superposition (input qubits) and put ancilla into $(|0\rangle - |1\rangle)/\sqrt{2}$
3. Apply the oracle Uf that maps |x⟩|y⟩↦|x⟩|y⊕f(x)⟩
4. Apply Hadamard gates to the n input qubits again.
5. Measure the n input qubits:
    - If the result is $|0\rangle^{\otimes n}$ (all zeros) with probability 1 → oracle is **constant**.
    - Otherwise (any non-zero bitstring appears) → oracle is **balanced**.

For this lab we will use the simple case n=4 (two input qubits) to illustrate both a constant and a balanced oracle.



Assignment:

1. Draw and solve the quantum circuit for the Deutsch–Jozsa algorithm with the constant oracle (f(x)=0) for n=2.
2. Draw and solve the quantum circuit for the Deutsch–Jozsa algorithm with the balanced oracle f (00) =f (10) =0 & f (01) =f (11) =1

Algorithm:

1. Import Qiskit and visualization tools.
2. Create a quantum circuit with n input qubits, 1 ancilla qubit, and n classical bits for measurement.
3. Initialize ancilla to |1> and apply H to all qubits.
4. Apply the chosen oracle Uf.
5. Apply H to the n input qubits again.
6. Measure the n input qubits into classical bits.
7. Simulate and plot the results.

Conclusion:

Experiment No. 7

Implement and run Grover's algorithm to search for a marked state.

Aim: To implement and run Grover's algorithm to search for a marked state.

Objectives:

The experiment focuses on:

- Application of Grover's algorithm for accelerating search in unstructured datasets. For example, searching a specific entry in a large unsorted database can be done in O(N) steps, unlike classical search that requires $O(\sqrt{N})$steps.
- Analysis of performance improvement achieved by Grover's search compared to classical exhaustive search. For instance, locating a marked state among 10^6 possibilities would take about 1000 iterations with Grover's algorithm, whereas a classical search could require checking up to a million entries.
- Exploration of practical applications of Grover's algorithm in cryptographic key recovery and related security scenarios. For example, breaking a symmetric key cipher like AES with 2^{128} keys would reduce to 2^{64} operations using Grover's algorithm, highlighting its significance in post-quantum cryptography.

Software Requirement:

- Python 3.x installed
- Qiskit installed: pip install qiskit
- Jupyter Notebook (recommended)

Theory:

Grover's Algorithm

Grover's Algorithm is a quantum search algorithm that provides a quadratic speedup over classical search methods. Classically, searching an unsorted database of N items requires $\mathcal{O}(N)$ queries in the worst case. In contrast, Grover's algorithm finds the desired item with high probability in only $\mathcal{O}(\sqrt{N})$ queries.

Problem Statement

Suppose we have a search space of $N = 2^n$ possible items, represented by computational basis states of n qubits:

$$\{|0\rangle, |1\rangle, \ldots, |N-1\rangle\}.$$

We are given an oracle function $f(x)$ such that

$$f(x) = \begin{cases} 1 & \text{if } x \text{ is the solution (the "marked state")} \\ 0 & \text{otherwise.} \end{cases}$$

The goal is to identify the marked state $|x^*\rangle$.

Algorithm Overview

Grover's Algorithm proceeds in the following steps:

1. **Initialization:** Prepare an equal superposition of all N basis states using Hadamard gates:
$$|\psi_0\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle.$$

2. **Oracle Query:** The oracle O_f flips the phase of the marked state:
$$O_f|x\rangle = (-1)^{f(x)}|x\rangle = \begin{cases} -|x\rangle & \text{if } x = x^*, \\ |x\rangle & \text{otherwise.} \end{cases}$$

3. **Amplitude Amplification (Diffusion Operator):** Apply the Grover diffusion operator, which inverts the amplitudes about their average:
$$D = 2|\psi_0\rangle\langle\psi_0| - I.$$

4. **Iteration:** Repeat the sequence of oracle and diffusion operators approximately
$$k \approx \frac{\pi}{4}\sqrt{N}$$
times to maximize the probability of measuring the marked state.

5. **Measurement:** Measure the quantum state. With high probability, the result will be the marked state $|x^*\rangle$.

Geometric Interpretation

Grover's algorithm can be understood geometrically as a rotation in a two-dimensional vector space spanned by:

$$|x^*\rangle \quad \text{(the marked state)}, \qquad |\psi_\perp\rangle \quad \text{(the superposition of all unmarked states)}.$$

Each Grover iteration (oracle + diffusion) rotates the state vector closer to the marked state. After $\mathcal{O}(\sqrt{N})$ iterations, the probability of observing the correct state is maximized.

Assignments on Grover's Algorithm

Assignment 1: Conceptual Questions

1. What is the role of the **oracle** in Grover's algorithm?
2. Explain why Grover's algorithm is said to provide a **quadratic speedup** compared to classical search.
3. Geometrically, how can we interpret each Grover iteration (oracle + diffusion) in the vector space of states?

Assignment 2: Worked Example (Small Database)

Consider a search problem with $N = 4$ items, represented by two qubits.

1. Prepare the initial state $|\psi_0\rangle$ as an equal superposition of all 4 states.
2. Suppose the marked state is $|11\rangle$. Show how the oracle O_f acts on the superposition.
3. Apply the diffusion operator and compute the new amplitudes of each state.
4. After one Grover iteration, what is the probability of measuring $|11\rangle$?
5. Compare this with the classical probability of finding the marked element in one query.

Implementation:

Result:

Conclusion:

Experiment No. 8

Implement and run the Quantum Fourier Transform (QFT) and Shor's algorithm.

Aim: To implement and run the Quantum Fourier Transform (QFT) and Shor's algorithm.

Objectives:

The experiment focuses on:

- Application of Quantum Fourier Transform for period finding and signal analysis. For example, QFT can efficiently determine the periodicity in a sequence, which is a key step in factoring large numbers.
- Analysis of Shor's algorithm for integer factorization and quantum speedup over classical methods. For instance, factoring a number like 15 using Shor's algorithm requires fewer quantum operations compared to classical trial division.
- Exploration of practical applications of Shor's algorithm in cryptography and cybersecurity. For example, Shor's algorithm can theoretically break RSA encryption by factoring large semiprime numbers, highlighting the importance of post-quantum cryptographic schemes.

Software Requirement:

- Python 3.x installed
- Qiskit installed: pip install qiskit
- Jupyter Notebook (recommended)

Theory:

Shor's Algorithm

Shor's algorithm is one of the most important quantum algorithms, famous for factoring integers in polynomial time. Since the best known classical algorithms require super-polynomial time for factoring the product of two large primes, cryptographic systems such as RSA rely on the difficulty of factoring. Shor's algorithm demonstrates that quantum computers can potentially break RSA by factoring efficiently.

The algorithm reduces the problem of factoring to the problem of **period finding**. If we can determine the period of the function

$$f(x) = a^x \bmod N,$$

then we can use it to efficiently find factors of N. Thus, the quantum part of Shor's algorithm focuses on solving the period finding problem.

The Problem: Period Finding

Given integers a and N (with $\gcd(a, N) = 1$), consider the function:

$$f(x) = a^x \bmod N.$$

The function is periodic, with period (or order) r, defined as the smallest positive integer such that:

$$a^r \bmod N = 1.$$

Finding this period r is the key to the algorithm. Once r is known, classical post-processing can yield non-trivial factors of N.

Quantum Solution Using Phase Estimation

To determine r, Shor used **Quantum Phase Estimation (QPE)** on the unitary operator:

$$U|y\rangle = |ay \bmod N\rangle.$$

The eigenstates of U are superpositions of computational basis states of the form:

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} \, |a^k \bmod N\rangle, \quad 0 \leq s \leq r-1.$$

The corresponding eigenvalues are:

$$U|u_s\rangle = e^{2\pi i s/r}|u_s\rangle.$$

If we apply QPE to U using the input state $|1\rangle$, we measure a phase of the form:

$$\phi = \frac{s}{r},$$

for a random integer s. By applying the **continued fractions algorithm**, we can recover the denominator r, the order of $a \bmod N$.

Modular Exponentiation

The most resource-intensive part of Shor's algorithm is implementing the modular exponentiation unitary:

$$U^{2^j}|y\rangle = |a^{2^j} y \bmod N\rangle.$$

This requires efficient circuits for modular multiplication and repeated squaring, which can be performed in polynomial time but are still the main bottleneck of the algorithm.

Factoring from Period Finding

Once the period r is obtained, we use the relation:

$$a^r \bmod N = 1.$$

If r is even, then:

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1).$$

Since $N \mid (a^r - 1)$, it follows that N must share a factor with one of the terms above. Thus, we compute:

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N),$$

which with high probability yield non-trivial factors of N.

Example: Factoring 15

Let $N = 15$. Choose $a = 7$ (coprime to 15). Compute the order r of 7 mod 15:

$$7^1 \equiv 7 \pmod{15}, \quad 7^2 \equiv 4 \pmod{15}, \quad 7^3 \equiv 13 \pmod{15}, \quad 7^4 \equiv 1 \pmod{15}.$$

So $r = 4$. Since r is even:

$$7^{r/2} - 1 = 7^2 - 1 = 48, \quad \gcd(48, 15) = 3,$$

$$7^{r/2} + 1 = 7^2 + 1 = 50, \quad \gcd(50, 15) = 5.$$

Thus, the factors of 15 are 3 and 5.

Steps of Shor's Algorithm

The steps of Shor's algorithm can be summarized as follows:

1. **Choose a random integer:** Select a random integer a such that

   $$1 < a < N$$

   where N is the composite number to be factored.

2. **Compute gcd:** Evaluate

   $$\gcd(a, N).$$

   If $\gcd(a, N) \neq 1$, then we have found a non-trivial factor of N, and the algorithm terminates.

3. **Quantum period finding:** If $\gcd(a, N) = 1$, we proceed to find the *order* r of a modulo N, defined as:

   $$r = \min\{k > 0 \mid a^k \equiv 1 \pmod{N}\}.$$

   This is achieved using a quantum computer via the Quantum Fourier Transform (QFT).

4. **Check order r:** If r is odd, or if $a^{r/2} \equiv -1 \pmod{N}$, then the process fails. In this case, choose a new a and repeat the procedure from step 1.

5. **Compute factors:** Otherwise, compute

$$\gcd\left(a^{r/2} - 1, N\right) \quad \text{and} \quad \gcd\left(a^{r/2} + 1, N\right).$$

These values give non-trivial factors of N.

Assignments:

Assignment 1: Conceptual Understanding

1. Explain the role of the **Quantum Fourier Transform (QFT)** in Shor's Algorithm.
2. Why does Shor's algorithm fail if the order r is odd or if $a^{r/2} \equiv -1 \pmod{N}$?
3. Compare the time complexity of Shor's algorithm with classical factorization algorithms.

Assignment 2: Numerical Example

1. Consider $N = 15$.
   (a) Choose $a = 2$ and check $\gcd(a, N)$.
   (b) Find the order r of a modulo N, i.e., the smallest positive integer r such that

$$2^r \equiv 1 \pmod{15}.$$

   (c) Use r to compute the possible non-trivial factors of 15 using

$$\gcd\left(2^{r/2} - 1, 15\right), \quad \gcd\left(2^{r/2} + 1, 15\right).$$

   (d) Verify that the factors are correct.

Implementation:

Result:

Conclusion: